

面向自动化数控编程的知识图谱增强大语言模型代码生成方法

First A. Author, *Fellow, IEEE*, Second B. Author, and Third C. Author, Jr., *Member, IEEE*

Abstract—大语言模型在语义理解和推理方面表现突出,为制造业通过自然语言驱动自动化编程提供了新思路。考虑到数控编程深度依赖工艺知识与安全规则,目前的生成手段在保证代码质量和生成受控方面还存在局限。本文介绍一种面向具身智能场景的通过知识图谱增强模型能力的数控代码生成方案,利用结构化知识引导生成过程。研究者将编程流程拆解为意图解析、参数补全和程序生成三个环节。在这一过程中,利用自反思机制提升对加工需求的理解精度,并在知识图谱的辅助下完成工艺参数的逻辑推理。通过模板化构建和多层校验,能够确保生成的代码符合工业生产的安全标准。实验结果证实,这套方案有效提高了模型处理复杂任务时的可靠程度,为工业领域应用大语言模型提供了实用参考。

Index Terms—大语言模型, 知识图谱, G 代码生成, 具身智能, 数控系统

I. INTRODUCTION

离散制造正朝着柔性化与高度定制化的方向快速发展,促使数控编程从传统的人工离线编程转变为任务导向的交互式自动处理。相比于依赖固定流程和人工核验的旧有模式,这种方式需要处理由语义需求和生产要素构成的复杂环境。即使加工意图一致,也需要针对不同的设备差异和现场状态,实时调整路径方案与安全限制。在多品种生产场景下,操作员通常仅提供有关加工特征和性能偏好的概括性指令。系统则必须将这些高层意图转化为包含坐标设置、刀具补偿及运动控制在内的底层细节,并严防干涉或超程等风险。此外,代码生成并非单次完成,而是通过持续的验证与反馈实现性能提升。由于此类任务涉及精准的意图解析、参数推理以及能够自我修正的逻辑循环,其复杂程度已明显高于传统规则驱动手段的处理范围。

解决这些难题需要构建具备深度理解和知识迁移能力的智能系统。与传统手段相比,大模型 (Large Language Model, LLM) 在逻辑推理及知识应用上表现更为优异,能够通过交互方式明确加工意图并收集参数,进而降低编程难度。即便如此,工业控制对安全性要求极高,模型产出的结果可能存在逻辑缺失或违反控制器规范的情况,影响最终程序的稳定性。为了增强生成过程的可控程度,一种

有效途径是将工艺规则和设备能力整理为结构化知识,并与生成机制相融合。在构建阶段,系统以工艺模板为核心,通过知识引导完成逻辑校验与代码输出。随后,利用多层验证策略自动识别并纠正潜在错误。通过这一套包含理解、生成与校验的完整流程,可以显著提升代码的可靠性及其在工程现场的实用性。

关于从自然语言到 G 代码的自动转化,现有的研究方向主要集中在两个领域。其中一类依靠预设规则与工艺模板,具备较高的稳定程度,但面对多样的表达需求时维护成本较高,难以适应复杂的加工约束。另一类则利用大型模型进行直接转化,虽然操作灵活,但在格式规范以及逻辑一致性上表现欠缺,容易产出参数超限的指令。为了改善这些状况,一些尝试侧重于通过复杂的提示策略和自反思机制引导模型优化输出。即便如此,这类手段通常需要设计冗长的提示内容,且缺乏对工艺细节与安全条件的统一表述,无法清晰展现从加工工艺到物理限制的层级关联。这也导致当结果出现偏差时,系统难以判定究竟是意图识别还是逻辑推理环节出现了偏差。以常见的车削加工为例,同一项任务受到刀具配置、加工余量、进给速度和空间运动限制等多重因素影响。任何微小的逻辑冲突都可能在后续环节中放大并引发生产风险。鉴于此,本文的研究目标是建立一套可控、可解释且安全可靠的自动化编程方案。该方案利用结构化数据定义工艺限制,结合多环节推理和分级校验,实现了从自然语言到加工代码的完整逻辑。

大模型在数控编程中的任务求解可以类比专业人员的逐步学习和调试过程。在应对复杂加工任务前,专业人员通常先建立对基础语法及加工循环的认知,随后才开始处理涉及工艺参数和作业限制的高级决策。这种由浅入深的方法能确保程序在受控状态下逐渐完善。类似的,如果要求模型直接根据模糊指令产出结果,容易在工艺逻辑上出现偏差。将任务拆解为多个可验证的中间环节,并在各阶段引入知识约束,则更有利于产出可靠的代码。例如在处理车削任务时,系统不仅要解析几何特征,还需推导出进给逻辑并执行参数校验。因此,面对复杂的制造环境,模型需要采用逐层细化的推理策略,在工程规则的引导下整合各项要素,最终产出安全的程序。这种逻辑在系统层面表现为由感知到决策再到执行的优化模式,为后续算法的协同设计提供了统一的解释框架。

基于上述分析,本文提出一种面向自动化数控编程的知识图谱增强大语言模型代码生成方法。如图 1 所示,该方案以具身智能为系统组织视角,在结构化知识的约束下建立了多环节 G 代码生成流程。系统首先利用自反思机制解析工艺意图并提取关键信息,随后通过交互补全策略完善加工参数。接着在知识引导下完成模板匹配和代码渲染,确保产出的指令符合工业标准。最后利用多级验证及纠错程序对结果进行评估,从而提升程序的可靠程度。这种方式将模型的语义理解与工程受控机制有机结合,为复杂加

This paragraph of the first footnote will contain the date on which you submitted your paper for review. It will also contain support information, including sponsor and financial support acknowledgment. For example, "This work was supported in part by the U.S. Department of Commerce under Grant BS123456."

The next few paragraphs should contain the authors' current affiliations, including current address and e-mail. For example, F. A. Author is with the National Institute of Standards and Technology, Boulder, CO 80305 USA (e-mail: author@boulder.nist.gov).

S. B. Author, Jr., was with Rice University, Houston, TX 77005 USA. He is now with the Department of Physics, Colorado State University, Fort Collins, CO 80523 USA (e-mail: author@lamar.colostate.edu).

T. C. Author is with the Electrical Engineering Department, University of Colorado, Boulder, CO 80309 USA, on leave from the National Research Institute for Metals, Tsukuba, Japan (e-mail: author@nrim.go.jp).

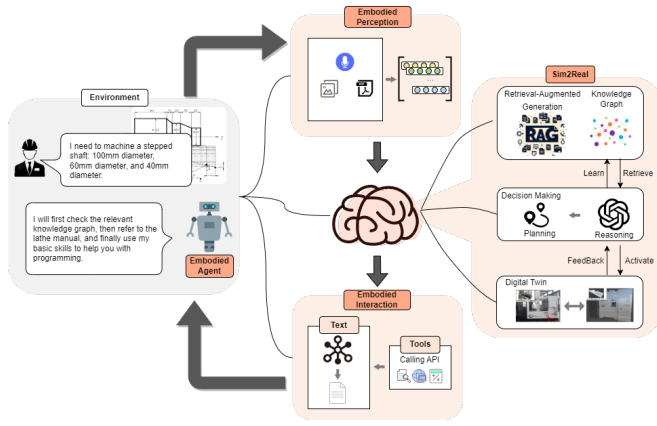


Fig. 1. Illustration of the proposed embodied-intelligence CNC programming framework empowered by a knowledge graph and LLMs. **Environment**: user instructions and machining context. **Embodied Perception**: multimodal parsing of task inputs. **Embodied Agent**: LLM-centered planning and reasoning supported by the knowledge graph (with optional documentation search). **Embodied Interaction**: tool and API invocation with code generation outputs. **Sim2Real**: digital-twin-based verification and feedback for iterative refinement.

工任务提供了可追溯且可应用的技术路径。

本文的主要贡献如下：

- 1) 提出知识图谱增强的数控代码生成框架，将生产要素进行结构化整合，并以完整的逻辑循环支撑自动化流程。
- 2) 设计意图识别与参数补全方案，实现对模糊指令的鲁棒理解，在降低沟通成本的同时确保参数完整。
- 3) 建立知识引导的模板匹配机制，在确保代码规范的前提下实现受控生成，并提供清晰的逻辑支撑。
- 4) 构建多级递进的验证与纠错体系，从多个维度约束输出质量，提升了加工过程的可靠程度。

II. RELATED WORKS

本节梳理大模型在智能制造与数控编程中的应用进展，评述受控代码生成的技术现状。通过分析现有研究的局限，明确本文的研究切入点。

A. LLMs for Manufacturing and CNC Programming

近年来，研究人员尝试将 LLM 整合进智能制造流程，使其作为连接人类意图、工程知识及可执行指令的语义接口 [1]。在设计和规划环节，模型能将开放需求转化为结构化数据，支持方案产出与知识复用 [2]。在生产现场，交互式助手被用于采集经验、规范描述和任务问答，降低了非专业人员参与流程的难度 [3]。随着智能体和工具调用技术的发展，模型开始承担调度及协调任务，能在对话中调用工艺库、参数表及仿真资源处理复杂制造工作 [4], [5]。

在数控编程领域，模型通过交互明确不完整指令并补全关键参数，辅助策略选取 [6], [7]。此外，部分研究尝试直接产出代码或程序框架，并结合后处理规则缩短开发周期 [8]。考虑到数控指令对语法、参数范围和安全有严格限制，直接生成结果仍可能出现逻辑缺失、参数错误或工艺冲突，且不易定位问题根源 [9], [10]。鉴于此，相关研究正从单纯的生成转向受控与验证，通过引入专业知识和工程限制提升工业可靠程度。这正是本文讨论的核心方向。

B. Knowledge-Driven Controllable G-code Generation and Verification

依据知识在生成流程中的功能差异，受控数控代码生成方法主要由规则模板、知识图谱引导及验证协同等模式组成。

规则模板方案将加工逻辑与控制器指令整合为规则库和参数化模板，通过将结构化数据填入预设位置直接产出数控程序 [11], [12]。Wang 等提出基于特征知识与几何建模的过程模型自动生成方法，用于快速构建多步加工过程模型 [13]。这种依靠明确规则的手段在可解释性以及符合工业标准方面表现突出，并能配合参数校验降低数据异常风险 [14]。不过此类内容高度依赖人工维护，面对不同设备和开放语义时调整成本显著增加，同时也难以处理复杂环境下的隐含限制 [15]。

知识图谱引导方案通过建立包含工艺规则、设备能力以及参数逻辑的知识体系，在生成环节利用图查询和推理技术完善规划内容，进而辅助复杂的流程设计及策略选取 [16]。相比于一次性生成的规则模板，该模式利用跨工序和跨资源的关联逻辑，增强了生产过程的一致度与可解释性。例如，Su 等提出基于制造知识图谱的决策支持框架，通过图推理辅助制造过程规划与信息补全 [17]；Meyers 等构建面向制造系统的知识图谱框架，以支持工艺规划及相关决策阶段的知识获取与一致性约束 [18]；此外，有研究表明将结构化知识与智能体 workflow 结合，在从任务理解到最终执行的各个环节利用数据引导决策方向，能够有效提升系统的协作效率 [19]。尽管具备这些优势，这类手段在建模与维护成本方面要求较高，且在不同控制器语法及工业标准的兼容与迁移上依然存在困难。

验证协同方案将校验、反馈及修复环节整合进生成流程，根据任务难度和风险等级自主选择验证策略，在确保质量的同时提升响应速度 [21]–[23]。Tao 等提出结合语法与格式约束的程序综合手段，将语法检查及参数范围核验等轻量化策略应用于简单场景，产出了高效的筛查结果 [24]。此外，Madaan 等提出 Self-Refine 框架，通过自反馈驱动的迭代细化方法，系统能在多轮生成中逐步定位偏差并修正结果，适用于复杂任务的渐进式处理 [25]。这种方式能够根据不同需求灵活配置校验流程，在处理效率和可靠程度之间达成平衡。

C. Research Gaps and Motivation

虽然相关研究在 LLM 驱动的任务理解、知识组织和受控生成方面有所突破，但数控编程的实际应用仍面临灵活性与安全性难以兼顾的冲突。规则模板手段更容易符合控制器标准且具备较好的解释性，但由于处理能力有限且维护代价较大，在面对多样化设备和开放语义需求时扩展受限。依靠模型直接产出的方式交互体验较好，但在严苛的语法要求及运动安全约束条件下，依然容易出现逻辑漏洞或参数失准等难以审计的问题。此外，知识图谱、模板渲染和验证修复通常被视为相互独立的环节，相关约束未能贯穿从意图理清、策略选取到代码生成及风险评估的完整流程。这使得故障定位及自动修正缺乏统一的技术支撑。

针对前述不足，本文构建一套自动化数控编程的统一框架。利用知识图谱对工艺规则、设备能力及参数逻辑进行结构化建模和约束，引导多环节推理和受控代码生成，并将分层验证与迭代修正机制整合进生成流程，以此提升产出结果的可追溯程度、验证效果及工程实用性。研究同时借鉴具身智能的系统架构，将数控设备作为执行单元，将

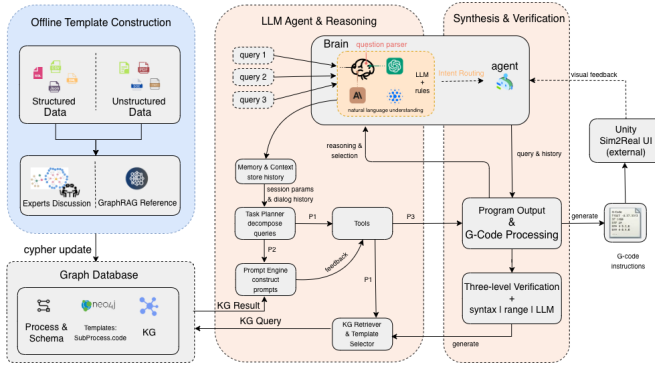


Fig. 2. Overall workflow of the LLM-KG CNC programming system.

Algorithm 1 Closed-loop knowledge-graph-enhanced G-code generation (P1–P3)

Require: User instruction u , context c , knowledge graph $\mathcal{G}$ (Neo4j), threshold τ

Ensure: Final G-code y and validation report R

- 1: **Offline:** curate templates and update Neo4j (*SubProcess.code*)
- 2: $\text{Normalize}(u', c') \leftarrow \text{Normalize}(u, c)$
- 3: **P3:** $(p, f, \text{conf}) \leftarrow \text{IntentParse}(u', c')$
- 4: **if** $\text{conf} < \tau$ **then**
- 5: $(p, f) \leftarrow \text{SelfReflect}(u', c', p, f)$
- 6: **end if**
- 7: **P1:** $C \leftarrow \text{KGQuery}(p, f, c'; \mathcal{G})$ (Cypher candidates)
- 8: **P2:** $\text{prompt} \leftarrow \text{AssemblePrompt}(u', c', C, \text{session})$
- 9: $m^* \leftarrow \text{SelectTemplate}(\text{prompt}, C)$
- 10: $y \leftarrow \text{Synthesize}(m^*, c')$ (execute *SubProcess.code*)
- 11: $R \leftarrow \text{Verify}(y)$ (syntax/range/LLM)
- 12: **while** $R.\text{pass} = \text{False}$ **do**
- 13: $(u', c') \leftarrow \text{Refine}(u', c', R)$
- 14: **P1–P3** repeat to update m^* and y
- 15: $R \leftarrow \text{Verify}(y)$
- 16: **end while**
- 17: **return** (y, R)

知识与模型作为认知支撑，通过交互澄清、生成、验证及反馈的迭代逻辑实现复杂任务的稳定求解。

III. METHODOLOGY

如 Fig. 2 所示，本文介绍一种基于知识图谱增强的数控编程方案。整体流程由四个相互衔接的阶段组成：1) workflow 组织与任务分解；2) Offline Template Construction；3) LLM Agent & Reasoning；4) Synthesis & Verification。这套方案结合多阶段协同推理与迭代校验逻辑，能够将自然语言形式的加工需求转化为安全可靠的可执行程序。本节将根据上述阶段依次介绍核心方法及实现细节。

A. Offline Template Construction

The offline stage aims to solidify domain expertise into reusable and auditable artifacts. Specifically, the process taxonomy (six major process categories and their sub-processes) is represented as *Process/SubProcess* nodes with *INCLUDES* relations, while executable templates

are stored in Neo4j as code strings in *SubProcess.code*. Template authoring is human-in-the-loop: experts review machining logic and controller constraints, optionally grounding decisions with documentation retrieval (e.g., GraphRAG references). The final templates are committed into the graph database via Cypher updates, providing stable candidates for online retrieval.

B. LLM Agent & Reasoning (P1–P3)

Given a user instruction u and runtime context c (machine/controller settings and session history), the agent first performs intent routing to distinguish machining tasks from knowledge queries. For machining tasks, it predicts the process type p and sub-process f with a confidence score conf . A self-reflection mechanism is triggered when $\text{conf} < \tau$ (with $\tau = 0.7$ in our implementation), prompting the model to re-examine potential keyword omissions and ambiguities.

The agent maintains a session memory that stores collected parameters and dialog history. The task planner decomposes the query and orchestrates tool calls. We use three staged plans to keep the pipeline controllable: P1 invokes tools and performs Cypher-based KG retrieval; P2 assembles a complete prompt by combining KG results, session parameters, and controller rules; and P3 drives the final reasoning for selection and output.

C. KG Retrieval & G-code Synthesis

In P1, the KG retriever queries Neo4j and retrieves candidate templates (i.e., *SubProcess.code*) associated with the identified process. The template selector ranks candidates using a multi-criteria strategy (LLM compatibility, parameter coverage, semantic consistency, and usage prior). After selecting m^* , the synthesis module executes the code stored in *SubProcess.code* to instantiate the corresponding process class and generate the G-code program y . Optional post-processing can be applied to ensure controller-specific formatting.

D. Synthesis & Three-level Verification with Visual Feedback

To ensure safety and executability, we adopt a three-level verification: (i) *syntax rules* to catch malformed lines and illegal command patterns, (ii) *parameter range checks* for critical fields (e.g., feed rate and spindle speed), and (iii) *LLM deep verification* for semantic and process-level issues, producing structured issues and improvement suggestions. The final verification score aggregates the three dimensions with safety prioritized:

$$s_{\text{overall}} = 0.5 s_{\text{safety}} + 0.25 s_{\text{eff}} + 0.25 s_{\text{std}}. \quad (1)$$

If verification fails, the report R provides evidence for refinement and triggers another iteration of P1–P3. In addition, an external Unity-based Sim2Real UI offers lightweight visual inspection; its visual feedback can assist the agent in iterative refinement when necessary.

REFERENCES

- [1] Z. Yu, P. Zhang, and J. Shi, "Transformation of industrial robotics with natural language models: Recent progress and future prospects," *Robotics and Computer-Integrated Manufacturing*, vol. 97, p. 103113, 2026.
- [2] Q. Xu *et al.*, "A large language model-enabled machining process knowledge graph construction method for intelligent process planning," *Advanced Engineering Informatics*, vol. 65, p. 103244, 2025.
- [3] S. Kernan Freire *et al.*, "Knowledge sharing in manufacturing using LLM-powered tools: user study and model benchmarking," *Frontiers in Artificial Intelligence*, vol. 7, p. 1293084, 2024.
- [4] C. I. Garcia *et al.*, "Framework for LLM applications in manufacturing," *Manufacturing Letters*, vol. 41, pp. 253–263, 2024.
- [5] J. Huang *et al.*, "Leveraging large language models for efficient scheduling in Human–Robot collaborative flexible manufacturing systems," *npj Advanced Manufacturing*, vol. 2, no. 1, p. 47, 2025.
- [6] M. Abdelaal, S. Lokadjaja, and G. Engert, "GLLM: Self-Corrective G-Code Generation using Large Language Models with User Feedback," *arXiv preprint arXiv:2501.17584*, 2025.
- [7] Y. Xie *et al.*, "Rapid generation method of process routes based on multi-agent collaboration with LLMs," *Advanced Engineering Informatics*, vol. 68, p. 103733, 2025.
- [8] H. Fan *et al.*, "AutoMEX: Streamlining material extrusion with AI agents powered by large language models and knowledge graphs," *Materials & Design*, vol. 251, p. 113644, 2025.
- [9] J. Umre, A. S. Parihar, and A. Gupta, "CSLLM: Code-Specific Large Language Models-A Survey," *Expert Systems with Applications*, p. 130991, 2026.
- [10] C. Chen *et al.*, "Integrating large language model and digital twins in the context of industry 5.0: Framework, challenges and opportunities," *Robotics and Computer-Integrated Manufacturing*, vol. 94, p. 102982, 2025.
- [11] R. Huang *et al.*, "An effective NC machining process planning method via integrating grammar knowledge with deep learning," *Expert Systems with Applications*, vol. 249, p. 123872, 2024.
- [12] W.-Y. Shi *et al.*, "Feature-based Recognition of CNC Processes and Automatic Program Generation," *Journal of Computers*, vol. 36, no. 6, pp. 187–203, 2025.
- [13] P. Wang *et al.*, "An automatic generation approach of process model based on feature knowledge and geometric modeling," *Advanced Engineering Informatics*, vol. 62, p. 102881, 2024.
- [14] K. Zhang *et al.*, "A novel solution of automated process planning techniques for STEP-NC manufacturing," *The International Journal of Advanced Manufacturing Technology*, pp. 1–28, 2025.
- [15] L. Guo *et al.*, "Ontology and production rules-based dynamic knowledge base construction methodology for machining process," *Journal of Manufacturing Systems*, vol. 77, pp. 1027–1044, 2024.
- [16] Z. Huang *et al.*, "mKGMPP: A multi-layer knowledge graph integration framework and its inference method for manufacturing process planning," *Advanced Engineering Informatics*, vol. 65, p. 103266, 2025.
- [17] C. Su *et al.*, "Knowledge graph-driven decision support for manufacturing process: A graph neural network-based knowledge reasoning approach," *Advanced Engineering Informatics*, vol. 64, p. 103098, 2025.
- [18] B. Meyers *et al.*, "Towards a knowledge graph framework for ad hoc analysis in manufacturing," *Journal of Intelligent Manufacturing*, vol. 35, no. 8, pp. 3731–3752, 2024.
- [19] Z. Qin and Y. Lu, "Knowledge graph-enhanced multi-agent reinforcement learning for adaptive scheduling in smart manufacturing," *Journal of Intelligent Manufacturing*, vol. 36, no. 8, pp. 5943–5966, 2025.
- [20] L. He and P. Jiang, "Manufacturing knowledge graph: a connectivism to answer production problems query with knowledge reuse," *IEEE Access*, vol. 7, pp. 101231–101244, 2019.
- [21] C. Zhang *et al.*, "Performance-driven closed-loop optimization and control for smart manufacturing processes in the cloud-edge-device collaborative architecture: A review and new perspectives," *Computers in Industry*, vol. 162, p. 104131, 2024.
- [22] A. Khan *et al.*, "A real-time closed-loop control system for adaptive robotic welding: a step toward intelligent welding," *The International Journal of Advanced Manufacturing Technology*, pp. 1–23, 2025.
- [23] R. Wang *et al.*, "Real-time process monitoring and closed-loop control on laser power via a customized laser powder bed fusion platform," *Additive Manufacturing*, vol. 66, p. 103449, 2023.
- [24] N. Tao *et al.*, "Grammar-obeying program synthesis: A novel approach using large language models and many-objective genetic programming," *Computer Standards & Interfaces*, vol. 92, p. 103938, 2025.
- [25] A. Madaan *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46534–46594, 2023.